MAHIP PAREKH — COMPLETE KNOWLEDGE BASE

SECTION 1: PERSONAL PROFILE

Name: Mahip Parekh Phone: +1 857 565 4266 Email: parekh.mahi@northeastern.edu LinkedIn: linkedin.com/in/mahip-parekh GitHub: github.com/mahip19 Location: Boston, MA (currently), originally from India

Education:

- Master of Science in Computer Science, Northeastern University, Khoury College of Computer Sciences, Boston, MA. January 2025 to December 2026. Currently in third semester.
- Bachelor of Engineering in Computer Engineering, Gujarat Technological University, India. July 2019 to July 2023.

Personal interests: Grinds his own coffee beans because he loves the fresh smell of coffee. Recently discovered he can cook almost as good as his mom. Enjoys playing video games with friends.

Family connection to MATLAB: Mahip's father completed his master's thesis in mathematics on linear control theory in 1994. His professor was an ISRO scientist who visited the USA for a conference and brought back a copy of MATLAB on a floppy disk. He passed it to Mahip's father who became the first student at his university to actually use MATLAB software. When Mahip told his father about his MathWorks interview, his father got goosebumps. This is a deeply personal connection to MathWorks.

SECTION 2: PROFESSIONAL EXPERIENCE

IDX India Pvt Ltd — Software Engineer (February 2023 to January 2024)

Company overview: IDX India builds B2B investor relations (IR) websites for publicly listed companies. These websites handle financial disclosures and shareholder communications.

Team: Product Support team for the company's internal product called "connect.ID drupal" which was built on top of Drupal CMS.

Daily responsibilities:

- Taking JIRA tickets from developers, client managers, and service team members
- Getting on calls with people from the build team, service team, and client managers to help troubleshoot and resolve product related issues
- Managing SSO access for client environments (dev, staging, production) as a team KPI
- Running post-upgrade QA checklists before giving green flag to hosting team to map live URLs
- Handling version conflicts, backend configurations, and platform upgrades across multiple client websites
- The company had three environments: dev, stage, and prod. Each required authentication. Mahip managed access assignments through JIRA tickets.

What Mahip did NOT do at IDX: He did not work directly with external customers. He did not do frontend development with React, Vue.js, or Next.js as primary work. He had some exposure to React and Vue.js by accompanying a team that was converting an internal issue-tracking tool from .NET to React and Vue, but this was not his official responsibility.

Drupal knowledge: Mahip does not remember how Drupal worked in detail. He was part of the product support team for connect.ID drupal. He would not be able to answer detailed technical questions about Drupal internals.

Important honesty note: Mahip's resume lists React.js, Vue.js, Next.js, and frontend optimization as things he did at IDX. This is inflated. His actual frontend experience at IDX was limited to informal involvement with the .NET to React conversion team. His real React experience came later in grad school building the Kambaz LMS.

Allsoft Solutions Pvt Ltd — Student Intern (June 2022 to August 2022)

Applied supervised learning algorithms including linear and logistic regression using Python, Scikit-learn, and Pandas. Achieved 82% prediction accuracy on customer churn datasets. Performed exploratory data analysis on 50,000+ row datasets.

SECTION 3: PROJECTS

Distributed File Storage System

Technologies: C++, Python, Linux, Multi-Paxos, Multi-Node Cluster
Course: CS 6650 Building Scalable Distributed Systems, December 2025
Partner: Samyak Shah

Architecture: Three layer system.

- Client Layer: Splits files into 1MB chunks, computes SHA-256 Content IDs (CIDs), orchestrates uploads and downloads.
- Metadata Layer (Chain Replication): Chain of 3 nodes (Head, Mid, Tail). Writes propagate down the chain and are acknowledged after reaching the Tail. Reads served exclusively by the Tail for strong consistency.
- Storage Layer (DHT Ring): Nodes arranged on a consistent hash ring. Each chunk stored on primary node and replicated to next k-1 nodes for fault tolerance. Replication factor = 2.

Work split: Mahip built the networking foundation including TCP socket wrapper classes (TCPServer, TCPClient) and the serialization/deserialization layer. Samyak focused on architectural planning, mapping what to implement next and how components would connect design-wise. Both worked across layers after initial split.

Key technical details of Mahip's networking code:

- Created wrapper classes around raw TCP socket calls (TCPServer.cpp and TCPClient.cpp)
- Used htonl() to convert message length to network byte order before sending
- Used ntohl() to convert back after receiving
- Implemented length-prefix protocol: send 4 bytes containing message length first, then send actual data
- Used while loops for partial sends and receives because TCP doesn't guarantee everything arrives in one call
- Used MSG_WAITALL flag for receiving the 4-byte length header
- Used SO_REUSEADDR socket option to allow server restart without waiting for old socket to release
- Used mutex on clients map for thread safety with concurrent connections
- SocketContext struct with valid flag for clean tracking of connected clients

Hardest bug: Messages sent from server to client were not deserializing correctly on the receiving end. Data looked fine on the sending side. Root cause was that there was no way to know where one message ended and the next began in the TCP stream. Solution was the length-prefix format where every message gets prepended with its length.

Performance results: 128+ concurrent client connections, sub-200ms latency for file operations, 99.5% availability during node failures, 3.2x throughput improvement.

Testing: Python testing harness for automated validation across multi-node Linux clusters. Tests included killing a storage node mid-operation and verifying file recovery from replica. Scalability tested with up to 50 concurrent clients. Throughput tested with file sizes from 10KB to 10MB.

Future work: Persistent storage with RocksDB or LevelDB. Dynamic configuration to avoid hardcoded IPs and ports.

Working style difference with partner: Samyak preferred to figure things out on the fly and not prioritize meetings. He kept saying they'd pull it off in the last week. Mahip preferred to plan ahead. Instead of forcing formal meetings, Mahip adapted by calling Samyak casually to collect ideas, then drafted action plans and set up skeleton code (interfaces, folder structures, starter tests) between meetings so they always had a clear path to follow.

Feature not implemented due to time constraints: When a dead node comes back online, it should automatically sync with the rest of the cluster and recover missed data. They decided to prioritize core functionality over this resync protocol due to hard deadline.

Calendar Event Manager

Technologies: Java, MVC, Swing, JUnit, Design Patterns
Course: Programming Design Paradigm

Built with a partner (teammate). Focus was on clean MVC architecture, SOLID principles, and design patterns (Factory, Adapter, Command). Goal was easy extensibility where new features could be added without modifying existing tested code. Included Java Swing desktop GUI with conflict detection, recurring events, and customizable calendar views. 200+ JUnit tests. 98% mutation test coverage.

Mutation testing: Framework that intentionally introduces bugs (mutations) into the code and checks if tests catch them. If a test still passes after a mutation, the test is weak. This was new to Mahip and fundamentally changed how he thinks about testing. Instead of asking "does this work" he started asking "what could go wrong and would my test catch it."

Testing disagreement with partner: Final phase required adding several new features and building a new GUI. Target was 100% mutation coverage. Teammate wanted to implement everything first and test at the end. Mahip was worried about cramming tests at the end. After going back and forth, Mahip suggested testing core behavior as they built each feature and saving edge cases for later. They made a checklist, rotated between implementing and testing (one person implements 2-3 features while other writes tests, then swap). Finished before deadline with 100% functionality and 98% coverage.

AI usage in this project: Mahip wrote all test cases in a plain text file first, then used AI to convert them into properly formatted JUnit test files. The thinking and test case design was his, AI handled the repetitive formatting.

Kambaz Learning Management System

Technologies: React, Node.js/Express, MongoDB, REST APIs, WebSockets
Course: Web Development

Full-stack replica of Canvas LMS. Supports course management, assignments, grading, real-time updates via WebSockets for 500+ concurrent users. This was the project where Mahip got comfortable with full-stack development end to end.

SECTION 4: BEHAVIORAL STORIES (FULL DETAIL)

Story A — Automation Script (IDX)

Context: The team was asked to produce a condensed report on what client websites are using what version of Drupal and the company's product. They had access to an internally hosted webpage listing all active client websites, but it didn't show version info. To get versions, one had to login to each website's backend using available access rights, navigate to admin > reports > status page, note the Drupal version (7, 8, or 9) and product version (1.7.1, 1.8.1, 1.8.2 etc) manually.

Mahip's action: Proposed automation. Built a script in Java using Selenium. The script opened the company's internal webpage listing client websites (three URLs: dev, stage, prod), used the dev URL to open each site, logged into the backend using a special user Mahip created for this task, navigated to the desired backend page (admin/report/status), extracted Drupal version and product version, and listed company name, website URL, Drupal version, and product version in an Excel sheet. Built three versions of the program for dev, stage, and production environments.

Discovery halfway through: Some old dev URLs were not working, nor their staging and production URLs. This meant clients had left or their environments were deleted. There was no single place tracking this. Mahip informed his manager who realized they could get version info AND have a solid list of active client websites to clear out old shutdown environments from the server.

Results: Dev had 400+ websites, 300+ staging websites, 270+ live production websites. Version details delivered for every active client website. Dead URLs identified for cleanup. Manager and team lead asked Mahip to document the script on Confluence, push to Bitbucket repo. Team lead and Mahip discussed improvements. The report was useful beyond their team — other teams asked to use it.

Script limitations and improvements: Initially the script wrote to Excel row by row which was slow. The laptop had to stay running for a week. After the initial run, Mahip optimized data insertion to bulk writes. His team lead handled the hosting side (moving it off the laptop to run independently) since the team lead had more experience with their infrastructure. Mahip was involved in the discussion but would be overstating it to say he did the hosting part himself.

Tools: Java, Selenium for browser automation.

Best used for: Ownership, Bias for Action, Invent and Simplify, Frugality, Deliver Results, Initiative, Leadership, Think Big.

Story B — Testing Disagreement (Calendar Project)

Full context: Final phase of semester-long calendar project. Had to add several new features and build an entirely new GUI desktop interface (previously command-line). Big scope increase. Goal was 100% mutation test coverage.

Disagreement: Teammate wanted to implement all features first then test. Mahip worried about cramming testing at end because mutation coverage requires thorough tests that catch intentional code mutations, which takes time and thought.

Resolution: Went back and forth for a while. Mahip stepped back and suggested aligning on priorities. Both speed and test quality were non-negotiable. Proposed middle ground: test each feature's core working behavior during implementation, save edge case testing for later. Never have pile of untested code but don't slow implementation chasing every edge case.

Work split: Made checklist of features. Mahip implements 2-3 features while teammate writes tests for completed ones. Then swap roles. At end, paired together for remaining edge case tests and bug fixes.

Result: 100% functionality, 98% mutation test coverage, finished before deadline.

Learning: When you disagree, stop debating the approach and align on what matters most first. Once agreed on priorities, compromise came naturally. Rotating responsibilities keeps things fair. Would suggest the priority-based approach sooner next time instead of extended debate.

Best used for: Disagree and Commit, Deliver Results, Are Right A Lot, Conflict Resolution.

Story C — Client Email Escalation (IDX)

Full context: After every website upgrade, the team had a prepared checklist of quality checks. It was Mahip's responsibility to go through those checkpoints precisely. Only when Mahip gave a green flag would the hosting team map the live URL of the company's website with the upgraded production website.

What happened: During post-upgrade QA checks, Mahip was working on a simple UI bug where the website administrator user was not able to "publish" an uploaded image. Should have been done by simply uploading image and clicking "publish" but it wasn't working.

The mistake: Mahip was doing final checks on a live website after it had been upgraded the previous night. To make sure no emails were fired, he disabled the client-email feature toggle button in the backend. But because the cache was full/stale, the toggle change was not reflected. Mahip assumed it was fine. He clicked the publish button multiple times to check, and each click

triggered emails to the actual client saying "image published, page updated." The client didn't know who was updating their site since they hadn't asked any service developer to do so.

Escalation: Issue got escalated. A meeting was called between the client manager of that client website and Mahip's team.

Mahip's response: Before the meeting, sat with his team and discussed what happened and what caused it. They figured out the cache of the client website had to be cleared from the server, which resolved the issue. In the meeting, Mahip explained the situation honestly, followed by a live demonstration of how a website acts when its cache is not removed from the server (doesn't show updated webpage, acts weirdly). The live demo was Mahip's own initiative to make the issue visible rather than just explained.

Result: Manager and team lead backed him up. Together they addressed the issue. Client was notified with the reason. Client was satisfied.

Learning: Taking a precaution isn't enough. You need to verify it actually worked. Mahip assumed the email toggle was off because he turned it off, but the cache meant it never took effect. Since then, always verifies changes are reflected before proceeding, especially on live environments.

Best used for: Earn Trust, Dive Deep, Failure/Mistake, Accountability, Safety/Quality.

Story D — Not Asking for Help (IDX)

Full context: This was Mahip's first professional corporate experience, coming straight from being a passionate college student. Had a fire within him that made him think he could tackle any problem himself without reaching out.

What happened: Was given view rights to the company's product code base and asked to understand the product. Team sat in adjoining desks in an open space. Mahip spent most available time scrolling through code, making notes. If something was unknown, he Googled it instead of asking teammates.

The confrontation: After two weeks, reported to manager and explained the product as he understood it. Manager was dissatisfied, said it was too little and Mahip was too slow. Manager knew exactly what was keeping him behind. Addressed it in a professional yet comforting tone saying "we are a team, and we got your back kid."

The change: Mahip changed his thinking and perception. Whenever he had time, he would text his senior manager to check if he was free, walk to his desk, sit beside him, and have friendly yet informative conversations about the product. Often got information about things he hadn't even discovered yet.

Result: Ramp-up accelerated dramatically. Went from slowly scrolling through code in isolation to understanding the product through the people who built it.

Learning: Asking for help isn't a weakness. It's how you learn faster and build relationships. Still has the instinct to try things independently but now sets a mental timer of 20-30 minutes. If stuck, reaches out. The instinct to be independent isn't bad — just needed to learn when to switch from solo mode to team mode. In college most work is individual and you're graded on what you know. Professional work is about team output.

Best used for: Learn and Be Curious, Weakness, Receiving Negative Feedback, Self Development, How You Handle Feedback.

Story E — Distributed File System (Northeastern)

Full context: Building Scalable Distributed Systems course. Partner was Samyak Shah. Built from scratch in C++. Mahip knew concepts like consistent hashing, chain replication, and thread pools theoretically from class but had never implemented any. Zero experience with TCP socket programming.

What Mahip did: Took on building the entire networking foundation. Started writing raw TCP socket code. Code got messy fast. Refactored into clean wrapper classes (TCPServer, TCPClient) that abstracted low-level details like byte ordering and connection handling.

The bug: Messages sent from server to client weren't deserializing correctly. Data looked fine on sending side but garbled on receiving end. After debugging, realized the fundamental problem was no way to know where one message ended and the next began in the TCP stream. Designed length-prefix format: prepend every message with its length (4 bytes, converted to network byte order using htonl) so receiver always knows how many bytes to read (using ntohl to convert back).

Result: 128+ concurrent connections, sub-200ms latency, 99.5% availability during node failures, 3.2x throughput improvement.

Technical concepts Mahip can explain:

- Consistent hashing: nodes and chunks on virtual ring, chunks go to nearest node clockwise, minimal data movement when nodes join/leave
- Chain replication: writes go Head to Mid to Tail, acknowledged only at Tail, reads from Tail only
- htonl/ntohl: converting between host byte order and network byte order for cross-machine communication
- Sockets and ports: port is like apartment number (which app gets the data), socket is the open connection between two machines
- TCP is a stream protocol with no message boundaries, hence need for length-prefix protocol

Best used for: Invent and Simplify, Learn and Be Curious, Dive Deep, Technical Challenge, New Skill.

Story F — SSO Access Management (IDX)

Full context: Company had three environments (dev, stage, prod). Access required authentication. Every person had users already made but needed specific client website access assigned. They needed to specify which environment in a JIRA ticket. This was product support team's KPI.

What happened: People would message Mahip on Teams, talk to him in cafeteria, or walk to his desk asking for access, saying they'd already talked to his manager. Mahip always respectfully denied and required a JIRA ticket. He explained his reasoning: needed to maintain a record of who has access to what site for what project.

Result: Maintained proper audit trail for production environment access. Manager fully supported this approach.

Best used for: Insist on Highest Standards, Have Backbone, Integrity, Process Discipline, Strengths.

Story H — Coding Community Contest (Undergrad)

Full context: Professor noticed Mahip's interest in competitive programming and asked him to lead the coding community as Programming Head. Two other core members handled socials and event management.

Regular activities: Organized events like hands-on problem solving sessions and intro-to-competitive-programming workshops.

Major achievement: Organized a full-scale competitive programming contest as part of college's annual tech fest. Mahip made problems himself at varying difficulty levels (easy for beginners, medium for intermediate, hard for competitive programmers), wrote test cases, hosted contest on CodeChef, coordinated with college computer lab faculty to book an entire floor of computers (since students in India don't typically bring laptops to college).

During the contest: Many students were doing CP for the first time. They knew the logic and approach but didn't know how to submit code on the platform. Mahip had to stay on his feet the entire time. Set up volunteers on the fly to help guide students through the submission process.

Promotion: The college had promotion teams that personally traveled to different colleges in neighboring cities to promote the festival. Mahip had done this himself as a freshman.

Result: 60-70 participants from multiple colleges across the region. Contest was a hit and became a recurring part of the tech fest.

What he'd do differently: Run a 10-minute platform walkthrough before the contest starts so fewer students struggle with submissions.

Best used for: Ownership, Think Big, Leadership, Deliver Results.

Story I — Coding Community Events (Undergrad)

Full context: As Programming Head, instead of just picking topics the team thought were interesting, Mahip spent time talking to junior students to understand what they were actually struggling with.

Discovery: Most juniors weren't stuck on concepts. They were stuck on practical things: how to start with competitive programming, how to turn their thinking into actual code whether it's a DSA problem or a project, and even non-technical stuff like how to build a professional LinkedIn profile.

Action: Designed entire event calendar around student needs. Every session came from listening first, not assuming. Prioritized topics based on how many students mentioned the same pain point.

Result: Attendance grew because students felt the events were relevant to them, not just generic tech talks.

Most popular events: "How to start CP" session and LinkedIn profile workshop due to immediate practical value.

Best used for: Customer Obsession, Think Customer, User Empathy.

Story J — Distributed Systems Teammate (Northeastern)

Full context: Same distributed systems project as Story E. Mahip and Samyak had very different working styles. Mahip prefers to plan things out before building. Samyak was more of a figure-it-out-on-the-fly kind of person. Samyak was the kind of student who'd spend the entire final week or couple of days before an exam pulling all-nighters, while Mahip preferred to prepare little by little beforehand. Samyak wasn't prioritizing project meetings because he had other work and kept saying they'd pull it off in the last week.

Mahip's approach: Instead of forcing formal weekly meetings, adapted to Samyak's style. Would call casually and ask how he'd like to implement a particular feature or what kind of fault tolerance he'd prefer. Collected ideas over time. By the time they finally sat down together, Mahip already had draft action plans, possible approaches, and designs ready. Would start implementing basic skeleton before every meeting: interfaces, folder structures, basic assumptions and tests. So when they worked together, they had a solid path instead of figuring things out on the fly.

Result: Project completed on time. 128+ concurrent connections, 99.5% availability. Healthy knowledge exchange due to this adapted working style. The instinct to plan ahead instead of cramming turned out to be the right call.

Learning: Sometimes being right isn't about winning the argument. It's about finding a way to get the right outcome without creating friction. Adapted process to partner's style while maintaining the outcome he believed was necessary.

Best used for: Are Right A Lot, Bias for Action, Deliver Results, Compromise, Working With Different Styles, Collaboration, Be Inclusive.

Story G — All Nighters (Backup Only)

Context: Had an assignment deadline approaching and was behind. Pulled two all-nighters to get it done. Delivered on time. Not his proudest planning moment but learned to start earlier and manage time better. Use only as last resort for pressure/deadline questions.

Undergrad Promotion Volunteer Story (Not fully developed)

During first year of undergrad, participated as a promotion volunteer for college's technical festival. Teams of volunteers would officially go to different universities and colleges in neighboring cities to promote the festival. Mahip's task was to visit a completely new college, approach random groups of students, visit classrooms and talk about the festival. Was shy and nervous initially, heart pumping fast. Pushed himself out of comfort zone. After a couple of awkward or failed conversations, had no more fear. Ended up selling more tickets than his share. His team covered the most campuses among all teams. First time pushing out of comfort zone.

Note: This story is weak for Amazon LPs but could work for "time you were outside your comfort zone" as a last resort.

SECTION 5: SKILLS AND TECHNICAL KNOWLEDGE

Programming Languages: Python, C++, C, Linux/Unix, Bash Scripting, Java, JavaScript, SQL
Tools and Technologies: Git/GitLab, Jenkins, Docker, JIRA, Agile/Scrum, CI/CD, Artifactory, Bitbucket, Confluence
Systems and Concepts: Multi-threading, Distributed Systems, Performance Optimization, Unit Testing, Data Processing, TCP Socket Programming, Consistent Hashing, Chain Replication, MVC Architecture, SOLID Principles, Design Patterns (Factory, Adapter, Command)
Databases: MySQL, PostgreSQL, MongoDB
Frameworks: React, Node.js/Express, Java Swing
Currently Learning: AWS, Machine Learning

Relevant Courses: Building Scalable Distributed Systems, Computer Architecture, Operating Systems, Data Structures and Algorithms, Object Oriented Programming, Software Engineering, Database Management Systems, Compilers, Programming Design Paradigm

Used MATLAB once in undergrad during Engineering Design course for projections and basic structural designs after working them out on paper first. Limited exposure.

SECTION 6: BOILERPLATE ANSWERS

Tell me about yourself (Amazon version, 60 seconds)

I'm Mahip, a Master's student in CS at Northeastern. Before grad school, I worked as a software engineer in India for a year on a product support team where I helped developers and client managers troubleshoot platform issues. Since starting my masters, I've built projects ranging from a distributed file storage system in C++ to a full stack learning management system. This semester I'm learning AWS and machine learning. I'm looking for a role where I can build things at scale and learn fast.

Tell me about yourself (MathWorks version, 90 seconds)

Hi, I'm Mahip. I'm a Master's student in Computer Science at Northeastern University in Boston. I'm currently in my third semester. Before coming to the US, I worked as a software engineer in India for about a year. I was on a product support team, where we spent a lot of time resolving issues, maintaining the product, and supporting upgrades. That experience taught me how important clear communication is, especially when you're helping other people troubleshoot and get unblocked. Since starting grad school, I've built projects across a wide range. I built a full stack learning management system. I also built a distributed file storage system in C++ focused on fault tolerance. And I built a Java based calendar management system, where I focused on clean MVC design and rigorous testing. This semester I'm also learning AWS and machine learning. I'm excited to build something meaningful at the intersection of both. Outside of tech, I'm into coffee and cooking. I grind my own beans because I love the fresh smell of coffee. And I recently realized I can cook almost as good as my mom. What drew me to the EDG program is the mix of strong engineering culture, collaboration, and continuous learning. I know as an intern I'll be focused on one team, but I really like that EDG supports long term growth over time. That's the kind of environment where I know I'll learn fast and contribute strongly.

Tell me about yourself (GM version)

I'm Mahip, a Master's student in CS at Northeastern, currently in my third semester. Before grad school, I worked as a software engineer in India for a year on a product support team. My job was to help maintain production systems, handle upgrades, run QA checks before sites went live, and

support internal developers when they hit platform issues. That experience gave me a strong appreciation for production reliability and quality. Since starting my masters, I've built a distributed file storage system in C++ focused on fault tolerance and data integrity, a Java based calendar system focused on clean architecture and rigorous testing, and a full stack learning management system. This semester I'm diving into AWS and machine learning. What excites me about this role is that the Data Labeling team builds the tools that create ground truth for autonomous driving. The quality of your labels directly affects how well the models perform, which directly affects safety on the road. That's not just software engineering, that's mission critical work.

Why Amazon?

Two things. First, the scale. Amazon operates systems that serve millions of users and I want to experience what it takes to build software at that level. Second, the ownership culture. Engineers at Amazon are expected to own their work end to end, not just throw code over a wall. That matches how I like to work. When I built the automation script at my previous job, nobody told me to do it. I saw the problem, proposed the solution, and owned the whole thing. I want to be in an environment where that's the norm, not the exception.

Why MathWorks?

MathWorks is the company that set the standard for simulation software. MATLAB and Simulink are foundational tools for engineers and scientists worldwide. What excites me is that any work I do here, even as an intern, eventually touches someone who's using these tools to push their field forward. I may not be the one doing the research, but I'm helping the person who is. That kind of indirect impact means a lot to me. As for EDG specifically, it's not just a "here's your desk, go code" kind of program. You're set up to learn deeply through supporting real users or working with a team on the product itself. Learning by doing meaningful work, not busywork.

Why GM?

Three things. First, the impact. GM is building the future of transportation with autonomous driving. The Data Labeling team creates the ground truth that teaches self driving cars to see the world. That's mission critical work where quality directly affects safety. Second, the tech breadth. TypeScript, React, Python, Golang, ML pipelines, workflow orchestration. Not a narrow intern role. Third, the culture. GM combines startup agility with enterprise scale.

What are your future goals?

Short term, solid industry experience, work on systems at scale, learn from senior engineers. Long term, own and lead technical projects end to end. Not just write code but understand the full picture. No rigid five year plan. Keep putting myself in environments where I'm learning fast and building things that matter.

Where do you see yourself in 5 years?

A senior engineer who can take an ambiguous problem, break it down, and drive it to completion. Someone the team trusts to own critical pieces. Don't know the exact domain yet. Focused on building a strong foundation and getting exposure to real world systems at scale.

What is success to you?

Knowing my work made a real difference. Whether that's a tool that saved my team weeks of work or helping a student submit their first CP solution. Not about titles. About leaving things better than I found them. Growth is also success. If I'm the same engineer a year from now, that's a failure.

How do you handle multiple tasks?

Prioritize based on urgency and impact. At IDX, juggled JIRA tickets, QA checks, and SSO requests daily. Check tickets in the morning, block time for focused work, handle quick requests in between. Key is being honest about what I can get done rather than saying yes to everything.

What are your strengths?

Take ownership without being told (automation script). Hold firm on process (SSO JIRA tickets). Communication skills learned from support role. Keep each to 15-20 seconds with a concrete example.

What is your weakness?

Tried to figure everything out myself instead of asking for help. Manager at first job called me out after two weeks. Changed completely. Still try things myself first but set 20-30 minute timer. Goal is team output, not proving I can do it alone.

How do you use AI/GenAI?

Two ways. Before coding: structure rough ideas into organized plans, generate checkpoints grouped by components. During implementation: only for redundant boilerplate. Example: wrote test cases in plain text, used AI to convert to formatted JUnit files. Thinking was his, AI handled repetitive typing. Never uses AI to make decisions or generate code he doesn't understand. If he can't explain every line, it shouldn't be in the project.

SECTION 7: COMPANY SPECIFIC KNOWLEDGE

MathWorks

- Products: MATLAB and Simulink
- CEO: Jack Little (co-founder)
- EDG Program: Engineering Development Group. Full-time engineers rotate through support, development, QA, UX, technical marketing before permanent team. Interns focus on one team.
- Core values: Continuous improvement, respect and invest, rational workplace, "do the right thing," generosity with time
- Private company, profitable every year since 1984, never had a layoff
- Culture: strong work-life balance, stability over hypergrowth
- Interview: Manager round is 25-30 min conversational behavioral. Feels "chill" but high rejection rate at this stage.

Amazon

- 16 Leadership Principles
- SDE intern interview: 2x60 min rounds, each 25-30 min coding + 20-30 min behavioral
- Each interviewer assigned 1-3 LPs
- 25% of candidates who pass technical bar get rejected on behavioral
- Say "I" not "we" for individual impact
- STAR method required

General Motors

- Vision: Zero crashes, zero emissions, zero congestion
- Brands: Chevrolet, GMC, Cadillac, Buick
- Cruise: Was GM's robotaxi subsidiary. Shut down in December 2024 after over $10 billion in losses and a 2023 pedestrian incident. GM pivoted to personal vehicle autonomy.
- Super Cruise: Hands-free driving on 20+ models, logging 10 million miles per month
- New autonomous strategy: Led by Sterling Anderson (former Tesla Autopilot chief). Hiring former Cruise engineers. Focus on hands-free then eyes-free then fully autonomous personal vehicles.
- Data Labeling team: Builds tools and pipelines for hybrid human/machine labeling. Creates ground truth for autonomous driving ML models. Tech stack: TypeScript, React, GraphQL, Python, Golang, Airflow.
- GM values: Customers, Excellence, Relationships, Seek Truth

- GM behaviors: Think Customer, Be Bold, Innovate Now, One Team, It's On Me, Look Ahead, Win With Integrity, Be Inclusive

Premera Blue Cross

- Healthcare company in Pacific Northwest
- Serves 2.8 million people
- Mission: Improving customers' lives by making healthcare work better
- Headquartered in Mountlake Terrace, WA
- SDE intern role requires hybrid work

SECTION 8: INTERVIEW HISTORY AND OUTCOMES

MathWorks EDG Intern

- Technical interview: Passed. Interviewer showed how EDG works (started Java, moved to support, now cybersecurity).
- Manager round: Completed. Answered 5-6 behavioral questions with no repeating stories. Interviewer was manager named Paul. Manager's feedback: "I don't really have anything in my mind to give you as feedback." Used the dad's MATLAB story when asked about long-term goals.
- Confidence was identified as an area to improve.
- Outcome: Received rejection. "Match not ideal."

Amazon SDE Intern

- Two 60-minute behavioral rounds completed.
- Questions asked: conflict, deep dive, compromise, new skill.
- Mahip lacked a strong compromise story going in.
- Outcome: Pending at time of last discussion.

General Motors SWE Intern (AI Training / Data Labeling)

- Recruiter outreach received.
- Behavioral round completed.
- Coding round completed on a Tuesday.

- Follow-up email sent the Monday after.
- Outcome: Pending at time of last discussion.

Premera Blue Cross SDE Intern

- Cover letter prepared with application questions answered.
- Status: Application submitted.

SECTION 9: LP AND COMPANY FRAMEWORK MAPPINGS

Amazon LP to Story Map

LP	Primary Story
Customer Obsession	I — Community Events
Ownership	A — Automation
Invent and Simplify	A — Automation or E — Distributed System
Are Right A Lot	J — Distributed Systems Teammate
Learn and Be Curious	D — Not Asking for Help
Insist on Highest Standards	F — SSO Access
Think Big	H — Coding Community Contest
Bias for Action	A — Automation
Frugality	A — Automation
Earn Trust	C — Client Escalation
Dive Deep	C — Client Escalation or E — Distributed System
Have Backbone; Disagree and Commit	B — Testing Disagreement
Deliver Results	B — Testing Disagreement or A — Automation

GM Behavior to Story Map

GM Behavior	Story
Think Customer	I — Community Events
Be Bold	B — Testing Disagreement
Innovate Now	A — Automation Script
One Team	J — Distributed Systems Teammate
It's On Me	C — Client Escalation
Look Ahead	J — Distributed Systems Teammate
Win With Integrity	F — SSO Access
Be Inclusive	J — Distributed Systems Teammate

SECTION 10: DELIVERY AND COMMUNICATION NOTES

Communication preferences: High-level simple language. No hyphens. STAR format for all stories. Answers kept to 60-90 seconds before pausing for interviewer to pull details. "I" not "we" for Amazon especially.

Confidence gap: Content quality is strong. Delivery confidence identified as area to develop. Subtle language softening (hedging, qualifying phrases, using "we" instead of "I") rather than story quality is the issue.

Interview introduction went from 2.5 minutes down to 60-90 seconds through iterative refinement over multiple drafts.

Key delivery rules:

- 60-90 seconds for initial answer, then stop
- Let interviewer ask follow-ups
- Don't rush, pause between STAR sections naturally
- Start with one sentence of context
- End with clear result or lesson, don't trail off
- If rambling past 2 minutes, cut to result
- Can say "let me think for a moment" before answering

- Be honest when you don't know something
- Numbers matter: 900+ websites, 128+ connections, 98% coverage, 60-70 participants